

LABORATOR 5 - LIMBAJE FORMALE SI AUTOMATE

MARIA PREDA

1. BREVIAR TEORETIC

2. CLASA LIMBAJELOR INDEPENDENTE DE CONTEXT

În cursurile/seminarele/laboratoarele anterioare am observat faptul că există anumite limbaje care nu se pot modela prin intermediul automatelor finite adică nu erau *limbaje regulate*.

Exemplu 2.1. *Următoarele limbaje nu sunt regulate:*

- $L_1 = \{a^n b^n \mid n \geq 0\}$
- $L_2 = \{ww^R \mid w \in \Sigma^*\}$

O parte dintre limbajele care nu fac parte din clasa *limbajelor regulate* fac parte dintr-o clasă numită **limbaje independente de context**. Cele două limbaje din *Exemplul 2.1* vor face parte din această familie.

Teoremă 2.2. *Fie CFL familia limbajelor independente de context. Atunci:*

$$\text{REG} \subset \text{CFL}$$

.

Observație 2.3. *Există mai multe familii de limbaje definite conform ierarhiei lui Chomsky:*

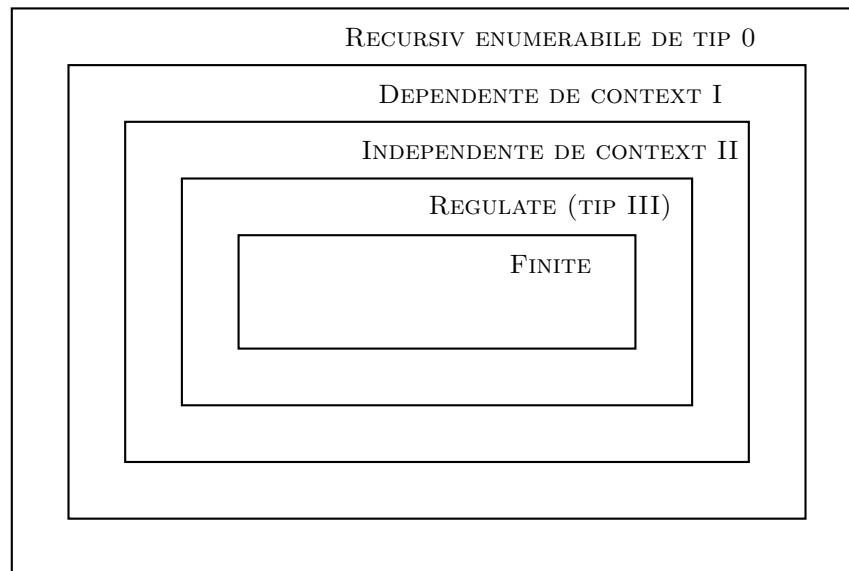


FIGURA 1. Ierarhia lui Chomsky. Sursa: A. Păun 2023, Curs 12: Decidabilitate. Limbaje Formale și Automate

Pentru a defini clasa limbajelor regulate ne-am folosit de automate finite, considerând că un limbaj este regulat dacă și numai dacă se poate construi un automat care să accepte toate cuvintele din acesta, și numai pe acestea. Vom încerca să definim astfel și clasa limbajelor independente de context.

Definiție 2.4. *Un limbaj independent de context este un limbaj acceptat de un automat push-down.*

Această definiție este într-o oarecare măsură circulară, pe viitor vom da o definiție mai riguroasă, utilizând gramatici.

2.1. Gramatici independente de context (CFG). Până acum am discutat despre limbaje din perspectiva automatelor. O altă modalitate de a defini limbajele independente de context este prin intermediul gramaticilor.

Definiție 2.5. *O gramatică independentă de context (CFG) este un tuplu (V, Σ, R, S) unde:*

- V este mulțimea simbolurilor neterminale;
- Σ este alfabetul terminal;
- R este mulțimea regulilor de producție de forma $A \rightarrow \alpha$, unde $A \in V$ și $\alpha \in (V \cup \Sigma)^*$;
- $S \in V$ este simbolul de start.

Observație 2.6. *Intuitiv, simbolurile neterminale sunt simboluri auxiliare care trebuie expandate, iar simbolurile terminale sunt caracterele care apar efectiv în cuvintele limbajului.*

Exemplu 2.7. *Fie gramatica:*

$$S \rightarrow aSb \mid \lambda$$

Aceasta generează limbajul:

$$L = \{a^n b^n \mid n \geq 0\}$$

Exemplu 2.8. *Fie gramatica:*

$$S \rightarrow aSa \mid bSb \mid \lambda$$

Aceasta generează palindroame peste alfabetul $\{a, b\}$.

Observație 2.9. *Gramaticile independente de context sunt echivalente ca putere de expresie cu automatele push-down nedeterminate.*

Teoremă 2.10. *Pentru orice gramatică independentă de context G există un PDA A astfel încât:*

$$L(G) = L(A)$$

și reciproc.

Observație 2.11. *Această echivalență este importantă deoarece:*

- gramaticile sunt mai ușor de folosit pentru descrierea limbajelor;
- automatele sunt mai potrivite pentru implementare.

Forma normală Chomsky. Pentru anumite demonstrații și algoritmi este util să lucrăm cu gramatici care au o formă mai restrictivă. Una dintre cele mai folosite astfel de forme este *forma normală Chomsky*.

Definiție 2.12. *O gramatică independentă de context este în forma normală Chomsky dacă fiecare regulă de producție este de una dintre formele:*

$$A \rightarrow BC$$

sau

$$A \rightarrow a$$

unde $A, B, C \in V$ și $a \in \Sigma$.

Observație 2.13. *Uneori se permite și regula $S \rightarrow \lambda$, dar doar dacă limbajul generat conține cuvântul vid. În acest caz, simbolul de start nu trebuie să apară în partea dreaptă a vreunei reguli.*

Intuitiv, forma normală Chomsky spune că fiecare pas de derivare trebuie să fie foarte simplu:

- fie un neterminat se transformă în două neterminale;
- fie un neterminat se transformă într-un terminal.

Astfel, orice derivare are o structură apropiată de un arbore binar.

Transformarea în forma normală Chomsky. Orice gramatică independentă de context poate fi transformată într-o gramatică echivalentă în forma normală Chomsky, eventual cu tratarea separată a cuvântului vid.

Pașii principali sunt următorii:

- (1) Adăugăm un nou simbol de start.

Dacă simbolul de start inițial este S , introducem un simbol nou S_0 și regula:

$$S_0 \rightarrow S$$

Acest pas este util mai ales dacă simbolul S apare în partea dreaptă a unor reguli.

- (2) Eliminăm regulile care produc λ .

Regulile de forma:

$$A \rightarrow \lambda$$

trebuie eliminate, cu excepția situației în care vrem să păstrăm cuvântul vid în limbaj.

Intuitiv, dacă un neterminat poate dispărea, atunci trebuie să adăugăm reguli care descriu toate situațiile în care acel neterminat este omis.

- (3) Eliminăm regulile unitare.

O regulă unitară este o regulă de forma:

$$A \rightarrow B$$

unde A și B sunt neterminale.

Aceste reguli nu produc direct terminale, ci doar redenumesc un neterminat. Ele pot fi eliminate prin copierea regulilor lui B la A .

- (4) Eliminăm simbolurile inutile.

Un simbol este inutil dacă nu poate contribui la generarea unui cuvânt terminal sau dacă nu poate fi atins pornind din simbolul de start.

- (5) Înlocuim terminalele din regulile lungi.

Dacă avem o regulă de forma:

$$A \rightarrow aB$$

aceasta nu este permisă în forma normală Chomsky, deoarece amestecă terminale cu neterminale.

Introducem un neterminat nou, de exemplu X_a , cu regula:

$$X_a \rightarrow a$$

și rescriem regula inițială:

$$A \rightarrow X_a B$$

- (6) Spargem regulile cu mai mult de două simboluri în partea dreaptă.

O regulă de forma:

$$A \rightarrow BCD$$

nu este permisă, deoarece are trei simboluri în partea dreaptă.

Introducem un neterminat nou, de exemplu X , și rescriem:

$$A \rightarrow BX$$

$$X \rightarrow CD$$

Observație 2.14. Scopul transformării nu este schimbarea limbajului, ci doar rescrierea gramaticii într-o formă mai ușor de analizat algoritmic.

Exemplu 2.15. Regula:

$$S \rightarrow aAB$$

nu este în forma normală Chomsky.

Introducem un neterminat nou X_a :

$$X_a \rightarrow a$$

Apoi rescriem:

$$S \rightarrow X_a AB$$

Cum partea dreaptă are trei simboluri, mai introducem un neterminat Y :

$$S \rightarrow X_a Y$$

$$Y \rightarrow AB$$

Acum toate regulile sunt de forma permisă:

$$A \rightarrow BC$$

sau

$$A \rightarrow a$$

Observație 2.16. Forma normală Chomsky este utilă mai ales atunci când vrem să verificăm algoritmic dacă un cuvânt aparține limbajului generat de o gramatică.

2.2. Automate push-down (PDA). Spre deosebire de automatele cu care am lucrat până acum, automatele push-down vin în plus cu o stivă de caractere, în care se scot și se adaugă anumite simboluri auxiliare la fiecare tranziție.

Observație 2.17. Automatele push-down pot fi privite ca o implementare operațională a gramaticilor independente de context. Stiva automatului joacă rolul de a memora simbolurile neterminale care trebuie expandate.

Definiție 2.18. Un automat push-down (PDA) este un tuplu $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ cu următoarea semnificație:

- Q : Mulțimea de stări
- Σ : Alfabetul de input
- Γ : Alfabetul stivei
- $\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \times Q \times \Gamma^*$, relație de tranziție.
- $q_0 \in Q$: starea inițială.
- $Z_0 \in \Gamma$: Simbolul de la capătul stivei.
- $F \subset Q$: mulțimea stărilor finale.

Sistemul de tranziție: Fie $(s, \sigma, \gamma, t, \tau) \in \delta$ o tranziție într-un automat push-down. Pornim dintr-o stare $s \in Q$ și vrem să consumăm din cuvânt un simbol $\sigma \in \Sigma \cup \{\lambda\}$, doar dacă la vârful stivei se regăsește simbolul auxiliar γ , astfel ajungând la starea t , după ce am scos de la vârful stivei simbolul γ (pop) și am adăugat (push) simbolul τ .

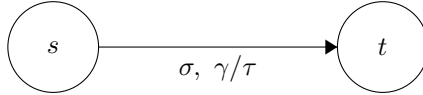


FIGURA 2. Tranziție într-un automat push-down

Observație 2.19. Pentru a simplifica notațiile, vom utiliza litere mici și cifre ca alfabet de input și majuscule pentru alfabetul stivei.

Observație 2.20. Simbolul Z_0 (uneori notat \$), se află mereu în stivă la început. Mereu va trebui să citim Z_0 de pe stivă la început și nu avem voie să nu îl punem la loc, decât când am finalizat de parcurs cuvântul.

Observație 2.21 (Instantanee). Pentru a descrie o etapă din parcurgerea unui cuvânt de către automat vom avea nevoie de un tuplu din $Q \times \Sigma^* \times \Gamma^*$, astfel memorând starea la care ne aflăm, simbolurile din cuvânt care mai trebuie consumate și configurația curentă a stivei.

Definiție 2.22 (Acceptarea cuvintelor). Automatele push-down au trei tipuri de acceptare:

- (1) Prin stări finale
 $T(A) = \{w \mid (q_0, w, Z_0) \vdash^* (q_f, \lambda, \alpha), q_f \in F\}$
- (2) Prin stiva vidă
 $N(A) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \lambda, \lambda)\}$
- (3) Prin stări finale și stiva vidă
 $L(A) = \{w \mid (q_0, w, Z_0) \vdash^* (q_f, \lambda, \lambda), q_f \in F\}$

Teoremă 2.23. Fie $\mathcal{T}_{PDA}, \mathcal{N}_{PDA}$ și $\mathcal{L}_{PDA}$ familiile limbajelor acceptate de automate pushdown cu stări finale, stiva vidă, respectiv stiva vidă și stări finale. Atunci:

$$\mathcal{N}_{PDA} = \mathcal{L}_{PDA} = \mathcal{T}_{PDA}$$

Definiție 2.24. Un automat push-down este determinist (DPDA) dacă și numai dacă respectă următoarele proprietăți Pentru orice $q \in Q, a \in \Sigma$ și $X \in \Sigma$:

- (1) $\delta(q, a, X)$ are cel mult un element
- (2) Dacă $\exists \delta(q, a, X)$, atunci $\nexists \delta(q, \lambda, X)$.

Teoremă 2.25. Fie $\mathcal{T}_{DPDA}, \mathcal{N}_{DPDA}$ și $\mathcal{L}_{DPDA}$ familiile limbajelor acceptate de automate pushdown deterministe cu stări finale, stiva vidă, respectiv stiva vidă și stări finale. Atunci:

$$\mathcal{N}_{DPDA} = \mathcal{L}_{DPDA} \subset \mathcal{T}_{DPDA}$$

3. INDICAȚII DE IMPLEMENTARE

3.1. Simulator de Automat Push-Down (PDA). Pentru PDA este cel mai natural să lucrăm cu configurații de forma:

(stare curentă, poziție în input, conținutul stivei)

Deoarece automatul poate fi nedeterminist, nu urmărim o singură execuție, ci o mulțime de configurații posibile.

Ideea de implementare. O soluție potrivită este BFS:

- pornim din configurația inițială;
- extragem pe rând o configurație;
- generăm toate configurațiile următoare posibile;
- acceptăm imediat când găsim o configurație care satisface condiția de acceptare.

O configurație poate fi memorată simplu ca un tuplu:

(state, index, stack)

unde **state** este starea curentă, **index** este poziția în șirul de intrare, iar **stack** este stiva curentă.

De exemplu:

config = ("q0", 0, ("Z0",))

Observație 3.1. Pentru a evita bucle infinite produse de tranziții λ , este important să memorăm configurațiile deja vizitate.

La fiecare tranziție:

- verificăm simbolul din input, dacă tranziția nu este cu λ ;
- verificăm simbolul din vârful stivei;
- facem *pop* pentru simbolul cerut;
- apoi adăugăm pe stivă șirul nou, de regulă în ordine inversă dacă folosim capătul listei sau al tuplului drept vârf al stivei.

Condiția de acceptare trebuie implementată exact conform cerinței:

- prin stare finală;
- sau prin stivă vidă;
- sau prin ambele.

3.2. Gramatică independentă de context $\rightarrow$ PDA. Această transformare nu este cerută pentru implementare completă, dar oferă o intuiție importantă despre modul de funcționare al automatelor push-down.

Ideea de bază. Pornim de la o gramatică $G = (V, \Sigma, R, S)$ și construim un PDA care:

- pornește cu simbolul S pe stivă;
- înlocuiește neterminalele folosind regulile din gramatică;
- consumă simbolurile terminale atunci când acestea coincid cu inputul.

Strategia. Pentru fiecare regulă $A \rightarrow \alpha$:

- dacă pe stivă avem A , putem face o tranziție λ care înlocuiește A cu α ;

Pentru fiecare simbol terminal a :

- dacă a apare în input și în vârful stivei, îl consumăm din ambele.

Intuiție. Automatul simulează o derivare:

- stiva conține ceea ce mai trebuie generat;
- inputul reprezintă ceea ce trebuie verificat;
- acceptăm dacă ajungem la stivă vidă și input consumat.

Observație 3.2. *Această abordare este nedeterministă, deoarece la fiecare neterminal putem avea mai multe reguli aplicabile.*

3.3. Generarea cuvintelor de lungime fixă dintr-o gramatică. O cerință posibilă este determinarea tuturor cuvintelor de lungime x generate de o gramatică independentă de context.

Ideea naturală este să pornim de la simbolul de start S și să aplicăm reguli de producție până când obținem forme formate doar din terminale.

$$S \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow w$$

unde $w \in \Sigma^*$.

Ideea de implementare. Putem privi fiecare formă sentențială ca pe o configurație. Pornim cu forma S și generăm noi forme aplicând câte o regulă pe un neterminal.

O variantă simplă este BFS:

- (1) pornim cu forma sentențială inițială S ;
- (2) extragem pe rând o formă;
- (3) dacă aceasta conține doar terminale, verificăm dacă lungimea este x ;
- (4) dacă mai conține neterminale, aplicăm regulile posibile și adăugăm formele noi în coadă.

Pseudocod.

```
queue <- [S]
visited <- empty set
result <- empty set

while queue is not empty:
    current <- pop first element from queue

    if current was already visited:
        continue

    add current to visited

    if current has only terminals:
        if length(current) == x:
            add current to result
            continue

    if number_of_terminals(current) > x:
        continue

    choose a nonterminal A from current

    for each rule A -> alpha:
        new_form <- current with A replaced by alpha
        add new_form to queue
```

Observație 3.3. *Pentru a evita generarea acelorași forme de mai multe ori, este util să memorăm formele deja vizitate.*

Observație 3.4. *Este important să oprim extinderea unei forme dacă aceasta conține deja mai mult de x simboluri terminale, deoarece nu mai poate genera un cuvânt de lungime exact x .*

Observație 3.5. Pentru gramatici care conțin reguli de tipul $A \rightarrow B$ sau $A \rightarrow \lambda$, pot apărea derivări foarte lungi sau chiar bucle. De aceea, în implementare se poate folosi și o limită pentru numărul de pași sau pentru lungimea formei sentențiale.